

Subset And Subsequence Difference

There's an important distinction here because **subset**, **subsequence**, and **substring/subarray** are often mixed up when learning backtracking.

1. Subset

A **subset** means:

Pick or don't pick each element. **Order does NOT matter.**

Example:

```
nums = [1, 2, 3]
```

All subsets:

```
[]  
[1]  
[2]  
[3]  
[1,2]  
[1,3]  
[2,3]  
[1,2,3]
```

Notice:

```
[1,2]  
[2,1]
```

are considered the **same subset**.

Backtracking idea

At every element, you have **2 choices**:

```
      []  
     / \  
  take 1  skip 1  
   [1]    []  
  /  \   /  \  
take2 skip2 ...
```

Typical code:

```
void solve(int index, vector<int>& nums, vector<int>& temp) {  
  
    if(index == nums.size()) {  
        ans.push_back(temp);  
        return;  
    }  
  
    // Take  
    temp.push_back(nums[index]);  
    solve(index + 1, nums, temp);  
    temp.pop_back();  
  
    // Don't take  
    solve(index + 1, nums, temp);  
}
```

So:

Subset = choose elements, order doesn't matter.

2. Subsequence

A **subsequence** is similar to a subset, but **order matters**.

You can remove elements, but you **cannot rearrange** the remaining elements.

For:

```
nums = [1, 2, 3]
```

Valid subsequences:

```
[ ]
[ 1]
[ 2]
[ 3]
[ 1, 2]
[ 1, 3]
[ 2, 3]
[ 1, 2, 3]
```

But:

```
[ 2, 1]
[ 3, 1]
[ 3, 2]
```

are **NOT** subsequences.

Why?

Because their order changed.

Original:

```
1 → 2 → 3
```

You can do:

```
1 → 3
```

by deleting 2 .

But you cannot do:

```
3 → 1
```

because you would have to rearrange the elements.

The interesting part

For an array with **distinct elements**, the generated choices look exactly like subsets:

```
[ 1, 2, 3]
```

produces:

```
[ ]
[ 1]
[ 2]
[ 3]
[ 1, 2]
[ 1, 3]
[ 2, 3]
[ 1, 2, 3]
```

So why do we distinguish them?

Because the **meaning is different**.

For example:

```
[ 1, 2, 3]
```

Subset:

```
{ 3, 1}
```

is perfectly fine and equivalent to:

```
{ 1, 3}
```

Subsequence:

```
[ 3, 1]
```

is NOT valid.

3. Subarray

This is another one you absolutely need to know.

A **subarray** must be **continuous**.

For:

```
nums = [1,2,3]
```

Subarrays:

```
[1]
[2]
[3]

[1,2]
[2,3]

[1,2,3]
```

But:

```
[1,3]
```

is **NOT** a subarray because 2 is between them.

The easiest way to remember

Think about what you're allowed to do:

Type	Delete elements?	Rearrange?	Must be continuous?
Subset	✓	✗*	✗
Subsequence	✓	✗	✗
Subarray	✗ inside selected range	✗	✓

*For subsets, order isn't considered at all.

Example that makes the difference obvious

Take:

```
[1, 2, 3, 4]
```

Consider:

```
[1, 3]
```

Subset?

✓ Yes.

We selected 1 and 3.

Subsequence?

✓ Yes.

1 comes before 3 in the original array.

Subarray?

✗ No.

Because 2 is between them.

Now consider:

```
[3, 1]
```

Subset?

✓ Yes.

The subset {1,3} is the same as {3,1}.

Subsequence?

✗ No.

Original order is:

1 → 2 → 3

not:

3 → 1

Subarray?

✗ No.

Obviously not contiguous.

Where backtracking comes in

This is the part you should focus on for LeetCode.

Subset / Subsequence

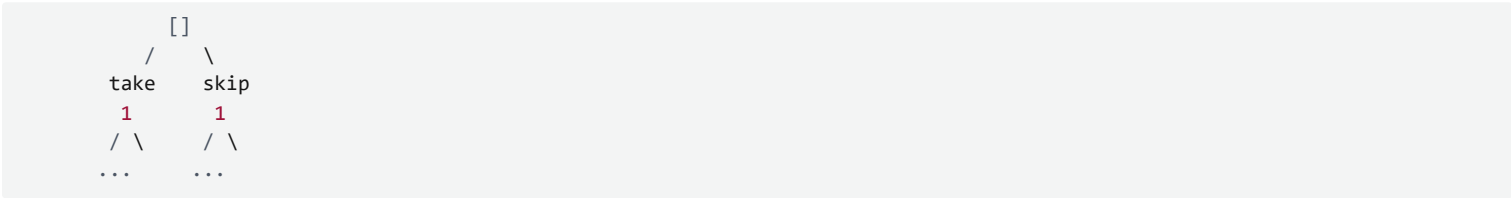
Usually you see:

index

moving forward:

0 → 1 → 2 → 3

At every index:



This gives approximately:

2^n

possibilities.

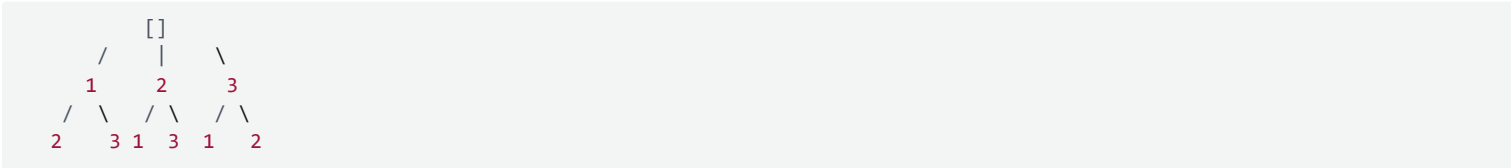
Permutation

This is different.

For:

[1,2,3]

you are allowed to choose any unused element next:



So:

Subset/Subsequence:
"Should I take this element?"

Permutation:
"Which unused element should I take next?"

That's the mental distinction you should keep.

One-line interview definitions

Subset: A selection of elements where **order doesn't matter**.

Subsequence: A sequence obtained by deleting some elements while **preserving their relative order**.

Subarray: A **contiguous** portion of an array.

And one correction to your wording: you wrote "subset, subset, subsequence". If you meant subset vs substring vs subsequence, then substring is mainly used for strings and means a contiguous part of a string.

Sure. The easiest way is to define all three using **contiguous**.

1. Subarray – CONTIGUOUS

A **subarray** is a group of elements that are **contiguous** in the original array.

[1, 2, 3, 4]

[2, 3]

[1, 2, 3]

[1, 3]

✓

✓

✗ not contiguous

Definition:

A subarray is a **contiguous** part of an array.

2. Subsequence – NOT NECESSARILY CONTIGUOUS

A **subsequence** is formed by deleting some elements while keeping the **original order**.

[1, 2, 3, 4]

[1, 3]

[1, 4]

[2, 4]

✓

✓

✓

They don't have to be next to each other.

[1, 2, 3, 4]

↑

↑

1

3

Definition:

A subsequence is an **ordered selection** of elements that **does not need to be contiguous**.

3. Subset – CONTIGUITY DOESN'T MATTER

A subset is simply a selection of elements where **order doesn't matter**.

[1, 2, 3]

{ 1, 3 }

{ 3, 1 }

✓

✓

← same subset

The elements don't need to be contiguous.

Definition:

A subset is a selection of elements where **contiguity and order don't matter**.

🔥 Remember this

SUBARRAY

→ MUST be contiguous

SUBSEQUENCE

→ MAY be non-contiguous, order preserved

SUBSET

→ Contiguity doesn't matter, order doesn't matter

Or even shorter:

Subarray

= contiguous

Subsequence

= order matters

Subset

= order doesn't matter

For backtracking, **subsets and subsequences often have the same take/skip recursion structure**, but the interpretation is different.